

ACPollerForAPS

Invoice pipeline

Multi-pipeline XML integration — AP Essentials, Dynamics AX, Comarch Optima

Product Manual

Version 2.0

Arondor

Activate your content

Contents

- 1. Introduction & concepts
- 2. Installation & startup
- 3. The configuration interface
- 4. Configuring a pipeline
- 5. Field mapping & output paths
- 6. Nested XML output (namespace, wrapper, sums)
- 7. Delivery transports & retry queue
- 8. ERP Providers (plugins)
- 9. Dashboard
- 10. Operations: logs, monitoring, troubleshooting

1. Introduction & concepts

ACPollerForAPS is an XML integration engine that transforms invoice and document data between systems by configuration, with no custom code. It reads an XML file, maps it to the structure a target system expects, and delivers it. Typical use: exchange data between AP Essentials (APS/ReadSoft) and one or more ERPs (Dynamics AX, Comarch Optima).

Pipelines

The core concept is the **pipeline: one input folder → one transformation → one output**. Each pipeline is autonomous, with its own watched folder, schedule and output. You can run several pipelines side by side — for example one for each direction of an exchange:

- **APS → Optima:** reads AP Essentials invoices, produces Comarch Optima XML, drops it where Optima imports it.
- **Optima → APS:** reads Optima exports, produces AP Essentials XML.

A pipeline does one thing end to end. Different rhythms or different formats simply mean different pipelines — there is no routing to configure.

Architecture

Component	Role
ACPollerForAPS.Core (DLL)	Shared model, transformation engine, validation, encryption, stats. Referenced by both the service and the UI — a single source of truth.
ACPollerForAPS.Service.exe	The Windows service that runs the pipelines.
ACPollerForAPS.UI.exe	The configuration interface (WPF).

The configuration file

The service and the UI share a single `settings.json`, structured as `{ "Pipelines": [ ... ] }`. The service reads it at startup and runs one worker per pipeline; the UI loads it automatically and edits it.

The service reads the configuration once, at startup. After a change through the UI, **restart the service** so it picks up the new config.

2. Installation & startup

Prerequisites

- .NET Framework 4.8 runtime on the target machine.
- Administrator rights (Windows service + Event Log).

Installer

Run the setup (`ACPollerForAPS-Setup-2.0.0.exe`). It asks for the install directory and lets you choose components — the **Service** and the **Configuration interface**. The installer stops any running service/UI first, copies the files, installs the service (which also creates the Windows Event Log source) and starts it. A default `settings.json` is deployed only if none exists, so the service can start on a fresh machine.

The uninstaller stops the service and the UI, removes the binaries and the service, but keeps `logs\`, `stats\`, `pending\` and `settings.json`, so configuration and history survive an upgrade.

3. The configuration interface

The window has a branded header, a toolbar (New, Open, Save, Save as, Validate on the left; Dark theme, Help, Manual on the right), a **pipeline selector** bar, and tabs: General, Schedule, Output, Dashboard.

The pipeline selector (top) lists all pipelines with Add / Remove / Copy. Pick one and the tabs below show that pipeline.

4. Configuring a pipeline

General tab

Field	Meaning
Pipeline name	A label; shown in the selector and the logs.

Input folder	Folder the service watches for incoming XML files.
Archive folder	Where processed files are moved after successful delivery.
Error folder	Where unreadable files are moved for review.
Archiving enabled	If off, sources are deleted instead of archived.
File filter	Which files to pick up (e.g. <code>*.xml</code>).
Stability check (ms)	Delay to confirm a file is fully written before processing.
Record path	XPath to a single record (invoice/document) in the input.

Schedule tab

Run interval (minutes or hours). Each run processes everything present.

Output tab

Holds the single output of the pipeline: format (Csv/XML), destination folder and file name, provider, sample XML, the XML/CSV format section, the transport, and the field mapping with preview.

Field	Meaning
Output format	Csv or Xml.
Output folder / file name	Destination and name. Tokens: <code>{date}</code> <code>{time}</code> <code>{guid}</code> <code>{batch}</code> .
Record path / Lines path	Record and repeating lines location in the input.
Provider	<code>mapping</code> (default) or a plugin DLL name — see section 8.
Sample XML	A representative input, stored in the config; feeds preview and path auto-completion.
Invoices per file	Max records per output file. <code>0</code> = one merged file.

5. Field mapping & output paths

The mapping grid defines the output fields, in order. Each field has:

Property	Meaning
Output path	The output column (CSV) or element path (XML). For nested XML, use <code>/</code> — e.g. <code>NAGLOWEK/SPRZEDAWCA/NIP</code> . Parent nodes are created and shared automatically.
Type	<code>text</code> · <code>amount</code> (decimals; Abs drops the sign) · <code>date</code> (Date in = input format).
Source	<code>fixed</code> = constant · <code>header / xpath</code> = record value · <code>line</code> = current line value · <code>sum</code> = aggregate over lines (see below).
Value / Path	Constant (fixed) / input XPath (others). Suggestions filtered by Source.
Dec. / Abs	Decimals / absolute value for amounts.
Date in	Input date format.
Only when (path) / equals	Conditional write: the field is written only when a line node equals a value.

Conditional mapping

Some formats produce, per record, lines of different kinds where some columns differ (e.g. Vendor vs Ledger lines). A field can be written only when a line node equals a value; otherwise it stays empty. Set *Only when (path)* and *equals* on the field.

Line numbering

A `line` field whose Path is the special value `#index` produces the line number (1, 2, 3...) — useful for an `LP` / position number.

6. Nested XML output key feature

The XML output is built from the field **Output paths** plus a small set of structural settings, so even deeply nested formats (like Comarch Optima) are produced by configuration alone.

Structure settings (XML format section)

Setting	Role	Optima example
Root element	The root tag.	<code>ROOT</code>
Namespace (xmlns)	Default namespace on the root. Empty = none.	<code>http://www.cdn.com.pl/optima/dokument</code>
Record element	Tag wrapping one record.	<code>DOKUMENT</code>
Line wrapper	Optional container around all lines.	<code>POZYCJE</code>
Line element	Tag wrapping one line.	<code>POZYCJA</code>

Structural tags come from these settings; the fields only fill the content. Header fields are placed under the record element, line fields under the line element, each at its Output path.

Sums (aggregation)

When the input has only lines but the target needs totals (e.g. Optima's `RAZEM_NETTO`), a field with Source `sum` and a line Path adds that value across all lines of the record. It respects *Only when*, so you can total a subset of lines (e.g. only debit lines).

Not covered by sums: values that combine two fields (e.g. gross = net + tax at total level) or per-rate VAT tables. Map those from the input if present, or handle them at implementation.

Reading nested input

Input paths can be nested too (e.g. `NAGLOWEK/SPRZEDAWCA/NIP`). Default namespaces in the input (as Optima uses) are neutralized automatically, so the paths stay simple.

Preview

Load a Sample XML, then click **Preview** to see the generated output. **Refresh paths** re-scans the sample. **Validate** (toolbar) checks every pipeline.

7. Delivery transports & retry queue

Each pipeline delivers its output through a transport, with configurable retry.

FS / FTPS / S3

Transport	Notes
FS	File written to the output folder.
FTPS	Explicit/implicit, certificate validation, encrypted password.
S3	Bucket + key prefix; custom endpoint for MinIO/S3-compatible; encrypted secret.

Credentials are encrypted with DPAPI (machine scope). Encrypt on the target machine; a secret encrypted elsewhere cannot be decrypted there.

Failed deliveries & automatic retry

If a delivery fails after all retries, the generated file is moved to a **pending queue** (`pending/<pipeline>/`) and the sources are archived normally — nothing is lost. At the start of every run the service retries the queued deliveries, so the file is delivered as soon as the target is reachable again. Pending deliveries appear in the run summary and the Event Log.

8. ERP Providers (plugins) advanced

Most targets are handled by the default `mapping` provider with no code. For a target whose logic the mapping cannot express, a DLL implementing `IErpExporter` can be dropped in the `providers/` folder next to the service; set the pipeline's Provider to its name. The service discovers it at startup; a broken DLL is logged and ignored. See PROVIDERS.md for the full guide.

9. Dashboard

The Dashboard tab gives an operations overview so you can see the integration is running well. It reads the run history the service records in the `stats\` folder (one line per run).

Element	Shows
Summary cards	Runs, files processed, outputs delivered, pending, error rate over the period.
Files per day	Volume bar chart; a bar is highlighted when that day had anomalies.
By pipeline	Per-pipeline table: runs, files, delivered, pending, failures, average duration.
Range / Refresh	Choose 7 / 30 / 90 days; refresh on demand.

The dashboard fills in as the service processes files. It reads `stats\` next to the service executable, so the UI sees it directly when both are installed together.

10. Operations

Logs

NLog writes to `logs\` next to the service exe, with daily + size rotation and purge. Each run logs a start line, per-file detail and a summary. Set the level to `Debug` in `NLog.config` for diagnostics.

Monitoring (Windows Event Log)

The service reports to a dedicated Windows event log **ACPollerForAPS** (source `ACPollerForAPS.Service`). Any enterprise monitor (Nagios, Zabbix, Centreon) reads it natively.

ID	Severity	Meaning
1000 / 1001	Information	Service started / stopped
1100	Information	Run completed OK
1200	Warning	Run completed with anomalies
1300	Error	Delivery failure / files still pending

Service lifecycle

```
sc start ACPollerForAPS
sc stop ACPollerForAPS
```

Restart the service after changing `settings.json` .

Troubleshooting

Symptom	Likely cause

Service installs but won't start (error 1064)	Missing <code>settings.json</code> next to the exe, or an invalid configuration. Check <code>logs\</code> and the Event Log.
Service runs but processes nothing	Interval too long, wrong input folder, or nothing in it.
XML paths don't match an input	For a namespaced format, paths stay simple (namespaces are neutralized) — check the path spelling against the sample.
Output XML has the wrong encoding	Set Encoding in the XML format section (UTF-8 by default).
FTPS/S3 secret cannot be decrypted	Encrypted on a machine other than the service's (DPAPI machine scope).
Files stuck in <code>pending/</code>	Delivery keeps failing; files are safe and retried each run. Check transport settings and the Event Log.
Dashboard is empty	No runs recorded yet, or the UI is not next to the service's <code>stats\</code> folder.